

quectel-BLEClient

版本：1.0

日期：2026-06-11

状态：受控/临时文件

文件保密级别：（请勾选 ☒ ）

绝密 ☐

保密 ☐

公开 ☒

文件管控表

文件变更记录			
修订日期	最新版本	修订内容	编制
2026-06-11	1	首次发行	Wells Li

上海移远通信技术股份有限公司（以下简称“移远通信”）始终以为客户提供最及时、最全面的服务为宗旨。如需任何帮助，请随时联系我司上海总部，联系方式如下：

上海移远通信技术股份有限公司
上海市闵行区田林路 1016 号科技绿洲 3 期（B 区）5 号楼 邮编：200233
电话：+86 21 5108 6236 邮箱：info@quectel.com

或联系我司当地办事处，详情请登录：<http://www.quectel.com/cn/support/sales.htm>。

如需技术支持或反馈我司技术文档中的问题，请随时登录网址：
<http://www.quectel.com/cn/support/technical.htm> 或发送邮件至：support@quectel.com。

前言

移远通信提供该文档内容以支持客户的产品设计。客户须按照文档中提供的规范、参数来设计产品。同时，您理解并同意，移远通信提供的参考设计仅作为示例。您同意在设计您目标产品时使用您独立的分析、评估和判断。在使用本文档所指导的任何硬软件或服务之前，请仔细阅读本声明。您在此承认并同意，尽管移远通信采取了商业范围内的合理努力来提供尽可能好的体验，但本文档和其所涉及服务是在“可用”基础上提供给您。移远通信可在未事先通知的情况下，自行决定随时增加、修改或重述本文档。

使用和披露限制

许可协议

除非移远通信特别授权，否则我司所提供硬软件、材料和文档的接收方须对接收的内容保密，不得将其用于除本项目的实施与开展以外的任何其他目的。

版权声明

移远通信产品和本协议项下的第三方产品可能包含受移远通信或第三方材料、硬软件和文档版权保护的相关资料。除非事先得到书面同意，否则您不得获取、使用、向第三方披露我司所提供的文档和信息，或对此类受版权保护的资料进行复制、转载、抄袭、出版、展示、翻译、分发、合并、修改，或创造其衍生作品。移远通信或第三方对受版权保护的资料拥有专有权，不授予或转让任何专利、版权、商标或服务商标权的许可。为避免歧义，除了正常的非独家、免版税的产品使用许可，任何形式的购买都不可被视为授予许可。对于任何违反保密义务、未经授权使用或以其他非法形式恶意使用所述文档和信息的违法侵权行为，移远通信有权追究法律责任。

商标

除另行规定，本文档中的任何内容均不授予在广告、宣传或其他方面使用移远通信或第三方的任何商标、商号及名称，或其缩略语，或其仿冒品的权利。

第三方权利

您理解本文档可能涉及一个或多个属于第三方的硬软件和文档（“第三方材料”）。您对此类第三方材料的使用应受本文档的所有限制和义务约束。

移远通信针对第三方材料不做任何明示或暗示的保证或陈述，包括但不限于任何暗示或法定的适销性或特定用途的适用性、平静受益权、系统集成、信息准确性以及与许可技术或被许可人使用许可技术相关的不侵犯任何第三方知识产权的保证。本协议中的任何内容都不构成移远通信对任何移远通信产品或任何其他软硬件、设备、工具、信息或产品的开发、增强、修改、分销、营销、销售、提供销售或以其他方式维持生产的陈述或保证。此外，移远通信免除因交易过程、使用或贸易而产生的任何和所有保证。

隐私声明

为实现移远通信产品功能，特定设备数据将会上传至移远通信或第三方服务器（包括运营商、芯片供应商或您指定的服务器）。移远通信严格遵守相关法律法规，仅为实现产品功能之目的或在适用法律允许的情况下保留、使用、披露或以其他方式处理相关数据。当您与第三方进行数据交互前，请自行了解其隐私保护和数据安全政策。

免责声明

- 1) 移远通信不承担任何因未能遵守有关操作或设计规范而造成损害的责任。
- 2) 移远通信不承担因本文档中的任何因不准确、遗漏、或使用本文档中的信息而产生的任何责任。
- 3) 移远通信尽力确保开发中功能的完整性、准确性、及时性，但不排除上述功能错误或遗漏的可能。除非另有协议规定，否则移远通信对开发中功能的使用不做任何暗示或法定的保证。在适用法律允许的最大范围内，移远通信不对任何因使用开发中功能而遭受的损害承担责任，无论此类损害是否可以预见。
- 4) 移远通信对第三方网站及第三方资源的信息、内容、广告、商业报价、产品、服务和材料的可访问性、安全性、准确性、可用性、合法性和完整性不承担任何法律责任。

版权所有 ©上海移远通信技术股份有限公司 2024，保留一切权利。

Copyright © Quectel Wireless Solutions Co., Ltd. 2024.

目录

1	模块概述	8
2	BLEClient 管理	8
2.1.	BLEClient	8
2.1.1.	接口概述	8
2.1.2.	函数原型	8
2.1.3.	参数描述	8
2.1.4.	返回值描述	8
2.1.5.	示例	8
2.2.	init	9
2.2.1.	接口概述	9
2.2.2.	函数原型	9
2.2.3.	参数描述	9
2.2.4.	返回值描述	9
2.2.5.	示例	9
2.3.	deinit	9
2.3.1.	接口概述	9
2.3.2.	函数原型	10
2.3.3.	参数描述	10
2.3.4.	返回值描述	10
2.3.5.	示例	10
3	BLE 基础控制	10
3.1.	start	10
3.1.1.	接口概述	10
3.1.2.	函数原型	10
3.1.3.	参数描述	10
3.1.4.	返回值描述	10
3.1.5.	示例	11
3.2.	stop	11
3.2.1.	接口概述	11
3.2.2.	函数原型	11
3.2.3.	参数描述	11
3.2.4.	返回值描述	11
3.2.5.	示例	11
3.3.	set_name_filter	11
3.3.1.	接口概述	11
3.3.2.	函数原型	11
3.3.3.	参数描述	11
3.3.4.	返回值描述	12
3.3.5.	示例	12
4	扫描操作	12

4.1.	set_scan_params.....	12
4.1.1.	接口概述.....	12
4.1.2.	函数原型.....	12
4.1.3.	参数描述.....	12
4.1.4.	返回值描述.....	13
4.1.5.	示例.....	13
4.2.	scan.....	13
4.2.1.	接口概述.....	13
4.2.2.	函数原型.....	13
4.2.3.	参数描述.....	13
4.2.4.	返回值描述.....	13
4.2.5.	示例.....	13
5	连接操作.....	14
5.1.	connect.....	14
5.1.1.	接口概述.....	14
5.1.2.	函数原型.....	14
5.1.3.	参数描述.....	14
5.1.4.	返回值描述.....	14
5.1.5.	示例.....	14
5.2.	disconnect.....	14
5.2.1.	接口概述.....	14
5.2.2.	函数原型.....	14
5.2.3.	参数描述.....	15
5.2.4.	返回值描述.....	15
5.2.5.	示例.....	15
5.3.	get_conn_id.....	15
5.3.1.	接口概述.....	15
5.3.2.	函数原型.....	15
5.3.3.	参数描述.....	15
5.3.4.	返回值描述.....	15
5.3.5.	示例.....	15
6	GATT 发现操作.....	16
6.1.	discover_services.....	16
6.1.1.	接口概述.....	16
6.1.2.	函数原型.....	16
6.1.3.	参数描述.....	16
6.1.4.	返回值描述.....	16
6.1.5.	示例.....	16
6.2.	discover_characteristics.....	16
6.2.1.	接口概述.....	16
6.2.2.	函数原型.....	17
6.2.3.	参数描述.....	17
6.2.4.	返回值描述.....	17

6.2.5.	示例	17
6.3.	discover_descriptors	17
6.3.1.	接口概述	17
6.3.2.	函数原型	17
6.3.3.	参数描述	17
6.3.4.	返回值描述	18
6.3.5.	示例	18
7	GATT 读操作.....	18
7.1.	read_char_by_handle	18
7.1.1.	接口概述	18
7.1.2.	函数原型	18
7.1.3.	参数描述	18
7.1.4.	返回值描述	18
7.1.5.	示例	19
7.2.	read_descriptor	19
7.2.1.	接口概述	19
7.2.2.	函数原型	19
7.2.3.	参数描述	19
7.2.4.	返回值描述	19
7.2.5.	示例	19
8	GATT 写操作.....	20
8.1.	write_char.....	20
8.1.1.	接口概述	20
8.1.2.	函数原型	20
8.1.3.	参数描述	20
8.1.4.	返回值描述	20
8.1.5.	示例	20
8.2.	write_char_no_rsp.....	20
8.2.1.	接口概述	20
8.2.2.	函数原型	21
8.2.3.	参数描述	21
8.2.4.	返回值描述	21
8.2.5.	示例	21
8.3.	write_descriptor.....	21
8.3.1.	接口概述	21
8.3.2.	函数原型	22
8.3.3.	参数描述	22
8.3.4.	返回值描述	22
8.3.5.	示例	22
9	MTU 操作	22
9.1.	exchange_mtu.....	22
9.1.1.	接口概述	22
9.1.2.	函数原型	23

9.1.3.	参数描述	23
9.1.4.	返回值描述	23
9.1.5.	示例	23
10	回调事件说明.....	23
10.1.	扫描结果事件.....	23
10.1.1.	事件类型	23
10.1.2.	回调数据格式.....	23
10.1.3.	字段说明	24
10.1.4.	示例	24
10.2.	连接事件	24
10.2.1.	事件类型	24
10.2.2.	回调数据格式.....	24
10.2.3.	示例	24
10.3.	Service 发现事件.....	25
10.3.1.	事件类型	25
10.3.2.	回调数据格式.....	25
10.3.3.	字段说明	25
10.3.4.	示例	25
10.4.	Characteristic 发现事件	25
10.4.1.	事件类型	25
10.4.2.	回调数据格式.....	26
10.4.3.	字段说明	26
10.4.4.	示例	26
10.5.	Descriptor 发现事件	26
10.5.1.	事件类型	26
10.5.2.	回调数据格式.....	26
10.5.3.	字段说明	27
10.5.4.	示例	27
10.6.	读结果事件	27
10.6.1.	事件类型	27
10.6.2.	回调数据格式.....	27
10.6.3.	字段说明	27
10.6.4.	示例	28
10.7.	Notify / Indicate 数据事件	28
10.7.1.	事件类型	28
10.7.2.	回调数据格式.....	28
10.7.3.	字段说明	28
10.7.4.	示例	28
10.8.	写结果事件	29
10.8.1.	事件类型	29
10.8.2.	回调数据格式.....	29
10.8.3.	字段说明	29
10.8.4.	示例	29
10.9.	MTU 事件.....	29

10.9.1. 事件类型	29
10.9.2. 回调数据格式	29
10.9.3. 字段说明	30
10.9.4. 示例	30
10.10. ATT 错误事件	30
10.10.1. 事件类型	30
10.10.2. 回调数据格式	30
11 常量定义	30
11.1. 扫描模式常量	30
11.2. 地址类型常量	31
11.3. Character 属性常量	错误!未定义书签。
11.4. 权限常量	错误!未定义书签。
11.5. 数据格式常量	错误!未定义书签。
11.6. 事件常量	错误!未定义书签。

1 模块概述

BLEClient 模块基于 BLE GATT Client 功能进行封装，提供 BLE 扫描、连接、服务发现、特征发现、描述符发现、读写特征值、读写描述符、MTU 交换以及事件回调等功能。

2 BLEClient 管理

2.1. BLEClient

2.1.1. 接口概述

创建一个新的 BLE GATT Client 对象。

2.1.2. 函数原型

```
client = quectel.BLEClient()
```

2.1.3. 参数描述

无。

2.1.4. 返回值描述

返回一个新的 BLEClient 对象实例。

2.1.5. 示例

```
import quectel
ble = quectel.BLEClient()
```

2.2. init

2.2.1. 接口概述

初始化 BLE GATT Client，并可注册 BLE Client 事件回调函数。

2.2.2. 函数原型

```
result = client.init(callback)
```

2.2.3. 参数描述

参数名	是 否 必填	数据类型	默认值	说明
callback	否	function	None	BLE Client 事件回调函数，接收 1 个参数： event_dict

2.2.4. 返回值描述

- 初始化成功返回 True
- 初始化失败返回 False
- 若 callback 不是可调用对象且不为 None，则抛出异常。

2.2.5. 示例

```
import quectel

def ble_client_cb(event):
    print("BLE Client event:", event)

client = quectel.BLEClient()

if client.init(ble_client_cb):
    print("BLE Client 初始化成功")
else:
    print("BLE Client 初始化失败")
```

2.3. deinit

2.3.1. 接口概述

反初始化 BLEClient 管理器并释放资源。

2.3.2. 函数原型

```
client.deinit()
```

2.3.3. 参数描述

无

2.3.4. 返回值描述

无返回值，若 BLE 未初始化，则抛出异常。

2.3.5. 示例

```
import quectel

client = quectel.BLEClient()
client.init()
client.deinit()
```

3 BLE 基础控制

3.1. start

3.1.1. 接口概述

开启 BLE GATT Client 功能。

该接口用于打开 BLE Client 功能开关。调用 scan()、connect() 等接口前，应先调用该接口。

3.1.2. 函数原型

```
client.start()
```

3.1.3. 参数描述

无。

3.1.4. 返回值描述

无返回值。 若执行失败，则抛出异常。。

3.1.5. 示例

```
import quectel

client = quectel.BLEClient()
client.init()
client.start()
```

3.2. stop

3.2.1. 接口概述

关闭 BLE GATT Client 功能。

3.2.2. 函数原型

```
client.stop()
```

3.2.3. 参数描述

无。

3.2.4. 返回值描述

无返回值，若失败抛出异常。

3.2.5. 示例

```
client.stop()
```

3.3. set_name_filter

3.3.1. 接口概述

设置 BLE 扫描名称过滤。

调用该接口后，扫描结果可按设备名称进行过滤。若传入空字符串，可用于取消名称过滤。

3.3.2. 函数原型

```
client.set_name_filter(name)
```

3.3.3. 参数描述

参数名	是否必填	数据类型	默认值	说明

name	是	str	-	需要过滤的 BLE 设备名称；为空字符串时表示取消过滤
------	---	-----	---	-----------------------------

3.3.4. 返回值描述

无返回值。
若执行失败，则抛出异常。

3.3.5. 示例

```
# 只扫描名称为 Uniknect_BLE_DEMO 的设备
client.set_name_filter("Uniknect_BLE_DEMO")

# 取消名称过滤
client.set_name_filter("")
```

4 扫描操作

4.1. set_scan_params

4.1.1. 接口概述

设置 BLE 扫描参数。
该接口用于配置扫描模式、扫描间隔、扫描窗口、扫描过滤类型以及本机地址类型。

4.1.2. 函数原型

```
ble.set_adv_params(min_interval, max_interval, adv_type, own_addrtype, channel_map)
```

4.1.3. 参数描述

参数名	是否必填	数据类型	默认值	说明
scan_mode	是	int	-	扫描模式，参考扫描模式常量
interval	是	int	-	扫描间隔，范围：4~16384
window	是	int	-	扫描窗口，范围：4~16384，且不能大于 interval
scan_type	是	int		扫描过滤类型，目前只支持 SCAN_FILTER_ALL
own_addrtype	是	int		本机地址类型，参考地址类型常量

4.1.4. 返回值描述

无返回值。 若参数非法或执行失败，则抛出异常。

4.1.5. 示例

```
import quectel

client = quectel.BLEClient()
client.init()
client.start()
client.set_scan_params(
    quectel.BLEClient.SCAN_PASSIVE,
    0x640,
    0x30,
    quectel.BLEClient.SCAN_FILTER_ALL,
    quectel.BLEClient.ADDR_PUBLIC
)
```

4.2. scan

4.2.1. 接口概述

开始或停止 BLE 扫描。

扫描到的设备会通过回调事件 EVT_SCAN_RESULT 返回

4.2.2. 函数原型

```
client.scan(start)
```

4.2.3. 参数描述

参数名	是否必填	数据类型	默认值	说明
start	是	bool	-	True 表示开始扫描，False 表示停止扫描

4.2.4. 返回值描述

无返回值。

若执行失败，则抛出异常。

4.2.5. 示例

```
# 开始扫描
client.scan(True)
```

```
# 停止扫描
client.scan(False)
```

5 连接操作

5.1. connect

5.1.1. 接口概述

连接指定 BLE 设备。

连接前通常需要先扫描到目标设备，并根据扫描结果中的 `addr` 和 `addr_type` 发起连接。

5.1.2. 函数原型

```
client.connect(addr_type, address)
```

5.1.3. 参数描述

参数名	是否必填	数据类型	默认值	说明
<code>addr_type</code>	是	<code>int</code>	-	对端设备地址类型，参考地址类型常量
<code>address</code>	是	<code>str</code>	-	对端设备地址字符串，例如 "9550c3c4eff9"

5.1.4. 返回值描述

无返回值。若执行失败，则抛出异常。

5.1.5. 示例

```
client.connect(quectel.BLEClient.ADDR_RANDOM, "9550c3c4eff9")
```

5.2. disconnect

5.2.1. 接口概述

断开当前 BLE 连接。

5.2.2. 函数原型

```
client.disconnect()
```


5.2.3. 参数描述

无。

5.2.4. 返回值描述

无返回值。 若执行失败，则抛出异常。

5.2.5. 示例

```
client.disconnect()
```

5.3. get_conn_id

5.3.1. 接口概述

获取当前 BLE 连接 ID。

连接建立后，后续服务发现、读写等接口需要使用 `conn_id` 参数。

5.3.2. 函数原型

```
conn_id = client.get_conn_id()
```

5.3.3. 参数描述

无。

5.3.4. 返回值描述

返回当前连接 ID。

5.3.5. 示例

```
conn_id = client.get_conn_id()
print("conn_id:", conn_id)
```

6 GATT 发现操作

6.1. discover_services

6.1.1. 接口概述

发现对端设备的 GATT Service。
可发现全部 Service，也可按 16-bit UUID 发现指定 Service。

6.1.2. 函数原型

```
client.discover_services(conn_id, uuid16)
```

6.1.3. 参数描述

参数名	是否必填	数据类型	默认值	说明
conn_id	是	int	-	BLE 连接 ID
uuid16	否	int	-	指定要发现的 16-bit Service UUID

6.1.4. 返回值描述

无返回值。
发现结果通过回调事件 EVT_SERVICE 返回。
若执行失败，则抛出异常。

6.1.5. 示例

```
# 发现全部 Service
client.discover_services(conn_id)

# 发现指定 Service
client.discover_services(conn_id, 0xFFFF0)
```

6.2. discover_characteristics

6.2.1. 接口概述

发现指定句柄范围内的 Characteristic。
通常需要先通过 discover_services() 获取 Service 的 start_handle 和 end_handle，再调用该接口发现该 Service 下的 Characteristic。

6.2.2. 函数原型

```
client.discover_characteristics (conn_id, start_handle, end_handle)
```

6.2.3. 参数描述

参数名	是否必填	数据类型	默认值	说明
conn_id	是	int	-	BLE 连接 ID
start_handle	是	int	-	起始句柄
end_handle	是	int	-	结束句柄

6.2.4. 返回值描述

无返回值。

发现结果通过回调事件 EVT_CHARACTER 返回。

若执行失败，则抛出异常。

6.2.5. 示例

```
client.discover_characteristics (conn_id, 16, 65535)
```

6.3. discover_descriptors

6.3.1. 接口概述

发现指定句柄范围内的 Descriptor。

通常用于查找 Characteristic 下的 CCCD 描述符，例如 UUID 为 0x2902 的 Client Characteristic Configuration Descriptor。

6.3.2. 函数原型

```
client.discover_descriptors(conn_id, start_handle, end_handle)
```

6.3.3. 参数描述

参数名	是否必填	数据类型	默认值	说明
conn_id	是	int	-	BLE 连接 ID
start_handle	是	int	-	起始句柄
end_handle	是	int	-	结束句柄

6.3.4. 返回值描述

无返回值。
发现结果通过回调事件 EVT_DESCRIPTOR 返回。
若执行失败，则抛出异常。

6.3.5. 示例

```
client.discover_descriptors(conn_id, 22, 25)
```

7 GATT 读操作

7.1. read_char_by_handle

7.1.1. 接口概述

根据 Attribute Handle 读取 Characteristic Value。

注意：

- 这里应传入 Characteristic 的 value_handle，不是 decl_handle。
- 当前封装固定为短读，不支持长包分段读取。

7.1.2. 函数原型

```
client.read_char_by_handle(conn_id, att_handle)
```

7.1.3. 参数描述

参数名	是否必填	数据类型	默认值	说明
conn_id	是	int	-	BLE 连接 ID
att_handle	是	int	-	Characteristic Value Handle

7.1.4. 返回值描述

无返回值。
读取结果通过回调事件 EVT_READ_RESULT 返回。
若执行失败，则抛出异常。

7.1.5. 示例

```
# 假设发现到 value_handle = 21
client.read_char_by_handle(conn_id, 21)
```

7.2. read_descriptor

7.2.1. 接口概述

读取指定 Descriptor 的值。
当前封装固定为短读，不支持长包分段读取。

7.2.2. 函数原型

```
client.read_descriptor(conn_id, att_handle)
```

7.2.3. 参数描述

参数名	是否必填	数据类型	默认值	说明
conn_id	是	int	-	BLE 连接 ID
att_handle	是	int	-	Descriptor Handle

7.2.4. 返回值描述

无返回值。
读取结果通过回调事件 EVT_READ_RESULT 返回。
若执行失败，则抛出异常。

7.2.5. 示例

```
# 读取 CCCD 描述符
client.read_descriptor(conn_id, 22)
```

8 GATT 写操作

8.1. write_char

8.1.1. 接口概述

向指定 Characteristic Value Handle 写入数据，使用有响应写。
该接口写入后，底层会返回写入结果事件 EVT_WRITE_RESULT。

注意：

- att_handle 应传入 Characteristic 的 value_handle。
- value 为十六进制字符串，value_len 为实际字节长度。
例如写入 "1234"，表示 2 字节数据，value_len 应填写 2。

8.1.2. 函数原型

```
client.write_char(conn_id, att_handle, value_len, value)
```

8.1.3. 参数描述

参数名	是否必填	数据类型	默认值	说明
conn_id	是	int	-	BLE 连接 ID
att_handle	是	int	-	Characteristic Value Handle
value_len	是	int	-	写入数据字节长度
value	是	str	-	写入数据内容

8.1.4. 返回值描述

无返回值。
若 value_len 为 0 或执行失败，则抛出异常。

8.1.5. 示例

```
# HEX 模式下写入 0x12 0x34
client.write_char(conn_id, 21, 2, "1234")
```

8.2. write_char_no_rsp

8.2.1. 接口概述

向指定 Characteristic Value Handle 写入数据，使用无响应写。

该接口适用于支持 Write Without Response 属性的 Characteristic。

注意：

- 对端 Characteristic 必须支持无响应写属性，否则可能写入失败或对端收不到数据。
- att_handle 应传入 Characteristic 的 value_handle。
- 无响应写通常不会有 ATT 层确认，应根据底层事件或对端接收情况判断是否成功。

8.2.2. 函数原型

```
client.write_char_no_rsp(conn_id, att_handle, value_len, value)
```

8.2.3. 参数描述

参数名	是否必填	数据类型	默认值	说明
conn_id	是	int	-	BLE 连接 ID
att_handle	是	int	-	Characteristic Value Handle
value_len	是	int	-	写入数据字节长度
value	是	str	-	写入数据内容

8.2.4. 返回值描述

无返回值。

若 value_len 为 0 或执行失败，则抛出异常。

8.2.5. 示例

```
# HEX 模式下无响应写入 0x01 0x02
client.write_char_no_rsp(conn_id, 21, 2, "0102")
```

8.3. write_descriptor

8.3.1. 接口概述

向指定 Descriptor Handle 写入数据。

常用于写入 CCCD 描述符以打开或关闭 Notify / Indicate。

常见 CCCD 写入值：

功能	HEX 值	value_len
关闭 Notify / Indicate	"0000"	2
打开 Notify	"0100"	2
打开 Indicate	"0200"	2

8.3.2. 函数原型

```
client.write_descriptor(conn_id, att_handle, value_len, value)
```

8.3.3. 参数描述

参数名	是否必填	数据类型	默认值	说明
conn_id	是	int	-	BLE 连接 ID
att_handle	是	int	-	Descriptor Handle
value_len	是	int	-	写入数据字节长度
value	是	str	-	写入数据内容

8.3.4. 返回值描述

无返回值。

若 value_len 为 0 或执行失败，则抛出异常。

8.3.5. 示例

```
# 打开 Notify
client.write_descriptor(conn_id, 22, 2, "0100")
```

```
# 打开 Indicate
client.write_descriptor(conn_id, 22, 2, "0200")
```

```
# 关闭 Notify / Indicate
client.write_descriptor(conn_id, 22, 2, "0000")
```

9 MTU 操作

9.1. exchange_mtu

9.1.1. 接口概述

发起 MTU 交换请求。

该接口应在 BLE 连接建立后调用。

9.1.2. 函数原型

```
client.exchange_mtu(mtu)
```

9.1.3. 参数描述

参数名	是否必填	数据类型	默认值	说明
mtu	是	int	-	期望协商的 MTU 值(23~247)

9.1.4. 返回值描述

无返回值。

MTU 协商结果通过回调事件 EVT_MTU 返回。

若执行失败，则抛出异常。

9.1.5. 示例

```
client.exchange_mtu(247)
```

10 回调事件说明

当调用 `client.init(callback)` 注册回调后，底层 BLE Client 事件会通过 MicroPython 调度机制回调到 Python 层。

回调函数接收一个字典对象 `event_dict`，其中至少包含以下字段：

字段名	说明
event	事件类型，参考 9.6 事件常量

根据不同事件类型，还可能附带其他字段。

10.1. 扫描结果事件

10.1.1. 事件类型

- BLEClient.EVT_SCAN_RESULT

10.1.2. 回调数据格式

```
{
```

```

    "event": event_id,
    "name": name,
    "addr": addr,
    "addr_type": addr_type,
    "rssi": rssi,
    "adv_type": adv_type,
    "adv_data": adv_data
}

```

10.1.3. 字段说明

字段名	说明
name	扫描到的设备名称
addr	设备地址
addr_type	设备地址类型
rssi	信号强度
adv_type	广播类型
adv_data	广播数据

10.1.4. 示例

```

def ble_client_cb(evt):
    if evt["event"] == quectel.BLEClient.EVT_SCAN_RESULT:
        print("发现设备:", evt.get("name"), evt.get("addr"), evt.get("addr_type"))

```

10.2. 连接事件

10.2.1. 事件类型

- BLEClient.EVT_CONNECTED
- BLEClient.EVT_DISCONNECTED

10.2.2. 回调数据格式

```

{
    "event": event_id,
    "conn_id": conn_id,
    "addr": addr
}

```

10.2.3. 示例

```

def ble_client_cb(evt):

```

```
if evt["event"] == quectel.BLEClient.EVT_CONNECTED:
    print("连接成功:", evt)
elif evt["event"] == quectel.BLEClient.EVT_DISCONNECTED:
    print("连接断开:", evt)
```

10.3. Service 发现事件

10.3.1. 事件类型

- BLEClient.EVT_SERVICE

10.3.2. 回调数据格式

```
{
    "event": event_id,
    "conn_id": conn_id,
    "start_handle": start_handle,
    "end_handle": end_handle,
    "uuid": uuid16
}
```

10.3.3. 字段说明

字段名	说明
start_handle	Service 起始句柄
end_handle	Service 结束句柄
uuid	Service 16-bit UUID

10.3.4. 示例

```
def ble_client_cb(evt):
    if evt["event"] == quectel.BLEClient.EVT_SERVICE:
        print("发现服务:", evt)
```

10.4. Characteristic 发现事件

10.4.1. 事件类型

- BLEClient.EVT_CHARACTER

10.4.2. 回调数据格式

```
{
  "event": event_id,
  "conn_id": conn_id,
  "uuid": uuid16,
  "handle": decl_handle,
  "value_handle": value_handle,
  "properties": properties
}
```

10.4.3. 字段说明

字段名	说明
uuid	Characteristic 16-bit UUID
handle	Characteristic Declaration Handle
value_handle	Characteristic Value Handle, 读写通信通常使用该句柄
properties	Characteristic 属性

说明:

handle 是 Characteristic 声明句柄, 主要用于描述特征声明本身; 实际读写 Characteristic Value 时, 使用 value_handle。

10.4.4. 示例

```
def ble_client_cb(evt):
    if evt["event"] == quectel.BLEClient.EVT_CHARACTER:
        print("发现特征:")
        print("uuid:", hex(evt["uuid"]))
        print("decl_handle:", evt["handle"])
        print("value_handle:", evt["value_handle"])
        print("properties:", evt["properties"])
```

10.5. Descriptor 发现事件

10.5.1. 事件类型

- BLEClient.EVT_DESCRIPTOR

10.5.2. 回调数据格式

```
{
  "event": event_id,
```

```
"conn_id": conn_id,
"uuid": uuid16,
"handle": handle
}
```

10.5.3. 字段说明

字段名	说明
uuid	Descriptor 16-bit UUID
handle	Descriptor Handle

10.5.4. 示例

```
def ble_client_cb(evt):
    if evt["event"] == quectel.BLEClient.EVT_DESCRIPTOR:
        print("发现描述符:", evt)
```

10.6. 读结果事件

10.6.1. 事件类型

- BLEClient.EVT_READ_RESULT

10.6.2. 回调数据格式

```
{
    "event": event_id,
    "conn_id": conn_id,
    "kind": kind,
    "handle": handle,
    "len": value_len,
    "value": value
}
```

10.6.3. 字段说明

字段名	说明
kind	读取来源，参考读结果来源常量
handle	被读取的 Attribute Handle
len	数据长度
value	读取到的数据

10.6.4. 示例

```
def ble_client_cb(evt):
    if evt["event"] == quectel.BLEClient.EVT_READ_RESULT:
        print("读取结果:", evt)
```

10.7. Notify / Indicate 数据事件

10.7.1. 事件类型

- BLEClient.EVT_NOTIFY
- BLEClient.EVT_INDICATE

10.7.2. 回调数据格式

```
{
    "event": event_id,
    "conn_id": conn_id,
    "handle": handle,
    "len": value_len,
    "value": value
}
```

10.7.3. 字段说明

字段名	说明
handle	产生 Notify / Indicate 的 Characteristic Value Handle
len	数据长度
value	收到的数据内容

10.7.4. 示例

```
def ble_client_cb(evt):
    if evt["event"] == quectel.BLEClient.EVT_NOTIFY:
        print("收到 Notify:", evt)
    elif evt["event"] == quectel.BLEClient.EVT_INDICATE:
        print("收到 Indicate:", evt)
```

10.8. 写结果事件

10.8.1. 事件类型

- BLEClient.EVT_WRITE_RESULT

10.8.2. 回调数据格式

```
{
    "event": event_id,
    "conn_id": conn_id,
    "which": which,
    "ok": ok
}
```

10.8.3. 字段说明

字段名	说明
which	写操作来源，参考写结果来源常量
ok	写入是否成功

10.8.4. 示例

```
def ble_client_cb(evt):
    if evt["event"] == quectel.BLEClient.EVT_WRITE_RESULT:
        print("写入结果:", evt)
```

10.9. MTU 事件

10.9.1. 事件类型

- BLEClient.EVT_MTU

10.9.2. 回调数据格式

```
{
    "event": event_id,
    "conn_id": conn_id,
    "mtu": mtu
}
```

10.9.3. 字段说明

字段名	说明
which	写操作来源，参考写结果来源常量
ok	写入是否成功

10.9.4. 示例

```
def ble_client_cb(evt):
    if evt["event"] == quectel.BLEClient.EVT_MTU:
        print("MTU:", evt["mtu"])
```

10.10. ATT 错误事件

10.10.1. 事件类型

- BLEClient.EVT_ATT_ERROR

10.10.2. 回调数据格式

```
{
    "event": event_id,
    "conn_id": conn_id,
    "att_err": att_err
}
```

11 常量定义

11.1. 扫描模式常量

常量名	值	说明
BLEClient.SCAN_PASSIVE	0	被动扫描
BLEClient.SCAN_ACTIVE	1	主动扫描

11.2. 地址类型常量

常量名	值	说明
BLEClient.ADDR_PUBLIC	0	公共地址
BLEClient.ADDR_RANDOM	1	随机地址

11.3. 扫描过滤常量

常量名	值	说明
BLEClient.SCAN_FILTER_ALL	0	扫描所有广播设备
BLEClient.SCAN_FILTER_WHITELIST	1	仅扫描白名单设备

11.4. 事件常量

常量名	值	说明
BLEClient.EVT_SCAN_RESULT	0	扫描结果事件
BLEClient.EVT_SCAN_DONE	1	扫描完成事件
BLEClient.EVT_CONNECTED	2	连接成功事件
BLEClient.EVT_DISCONNECTED	3	连接断开事件
BLEClient.EVT_SERVICE	4	发现 Service 事件
BLEClient.EVT_INCLUDE	5	发现 Include Service 事件
BLEClient.EVT_CHARACTER	6	发现 Characteristic 事件
BLEClient.EVT_DESCRIPTOR	7	发现 Descriptor 事件
BLEClient.EVT_READ_RESULT	8	读取结果事件
BLEClient.EVT_NOTIFY	9	Notify 数据事件
BLEClient.EVT_MTU	10	MTU 交换事件
BLEClient.EVT_INDICATE	11	Indicate 数据事件
BLEClient.EVT_WRITE_RESULT	12	写入结果事件
BLEClient.EVT_ATT_ERROR	13	ATT 错误事件

11.5. 写结果来源常量

常量名	值	说明
-----	---	----

BLEClient.WRITE_CHAR	0	有响应写 Characteristic
BLEClient.WRITE_CHAR_NO_RSP	1	无响应写 Characteristic
BLEClient.WRITE_DESC	2	写 Descriptor

11.6. 读结果来源常量

常量名	值	说明
BLEClient.READ_BY_HANDLE	0	按 Handle 读取 Characteristic
BLEClient.READ_BY_UUID	1	按 UUID 读取 Characteristic
BLEClient.READ_DESC	2	读取 Descriptor

12 BLE Client 使用流程示例

12.1. 基本流程

BLE GATT Client 一般使用流程如下：

1. 创建 BLEClient 对象
2. 调用 init() 初始化并注册回调
3. 调用 start() 打开 BLE Client
4. 调用 set_scan_params() 配置扫描参数
5. 调用 scan(True) 开始扫描
6. 在扫描回调中获取目标设备 addr 和 addr_type
7. 调用 scan(False) 停止扫描
8. 调用 connect() 连接目标设备
9. 连接成功后获取 conn_id
10. 调用 discover_services() 发现服务
11. 调用 discover_characteristics() 发现特征
12. 根据 value_handle 进行读写
13. 如需 Notify / Indicate, 发现 Descriptor 后写入 CCCD

13 使用注意事项

1.read_char_by_handle()、write_char()、write_char_no_rsp() 应使用 Characteristic 的

value_handle, 不是 decl_handle。

2.discover_characteristics() 前通常需要先通过 discover_services() 获取服务句柄范围。

3.打开 Notify / Indicate 时, 需要先找到对应的 CCCD 描述符句柄, 一般 UUID 为 0x2902。

4.写 CCCD 时:

- 打开 Notify: "0100"
- 打开 Indicate: "0200"
- 关闭 Notify / Indicate: "0000"

5.当前封装中的读接口固定为短读, 不支持长数据分包读取。

6.当前封装中的写接口固定为短写, 不支持长数据分包写入。

7.当前 BLE Client 只支持 HEX 数据格式, value_len 表示实际字节数, 不是 HEX 字符串长度。

例如 "1234" 的 value_len 为 2。

8.如果对端 Characteristic 不支持 Write Without Response 属性, 调用 write_char_no_rsp() 可能无法被对端正常接收。

9.扫描到设备后, 建议使用扫描结果中的 addr_type 进行连接, 避免 Public / Random 地址类型不匹配导致连接失败。